

CS 514 – Database Theory and Implementation

Final Database Product Project

Project Overview

Working in teams of 4-6 students, you will design, implement, and present an original database-driven software application. Your project may be an original idea or a significant enhancement of an existing application. The goal is to apply the concepts learned throughout the semester while creating a functional, well-designed database product.

This project is an opportunity to strengthen your technical skills, collaborate with your classmates, and develop a project that can become part of your professional portfolio. Treat this project as a professional software development effort. Focus on creating a well-designed database application, writing clean and maintainable code, documenting your work thoroughly, and collaborating professionally with your teammates. Feel free to consult with your teaching assistants and instructor throughout the project for guidance and feedback.

Have Fun and Be Creative!

Databases are at the core of nearly every modern software system. Use this project as an opportunity to solve a real-world problem, apply the concepts learned throughout the course, and create a database product that showcases your creativity and technical abilities. Some of today's most successful software products began as class projects or ideas developed by small teams of students. Think creatively about solving a real-world problem that interests you. With continued development, your database product could evolve into an open-source application, research project, startup company, or commercial software product. Build something meaningful that you would be proud to demonstrate during internships, job interviews, or future career opportunities. You never know where a great idea may lead, so challenge yourselves to think creatively, work together, have fun and enjoy the process.

Project Objectives

- Design a well-structured database.
- Develop a functional database-driven application.
- Apply SQL and/or NoSQL and database programming concepts.
- Work effectively as a software development team.
- Document and professionally present your work.

Applying Course Concepts

Your project should incorporate course concepts learned throughout the course where appropriate. Some examples include:

- Entity-Relationship (ER) modeling
- Relational database design
- Primary and foreign keys
- Normalization
- SQL queries (SELECT, INSERT, UPDATE, DELETE)
- Joins, aggregate functions, and subqueries
- Constraints, views, or stored procedures
- Transactions
- Database programming

Not every project is expected to use every database feature. Select the concepts that best support your application's design and functionality.

Team Expectations

Every team member is expected to contribute equally throughout the semester.

- Participate in database design, application development, testing, documentation, presentations, etc.
- Maintain meaningful GitHub contributions.
- Participate in both the Draft and Final presentations.
- Complete a confidential peer evaluation for each teammate.

Confidential Peer Evaluation

Each student will confidentially evaluate every teammate in the comments section of their Phase 1 Draft and Phase 2 Final submission. If you do not evaluate your teammates, points will be deducted.

Criterion	Scale	Weight
Attendance & Participation	1–10	20%
Technical Contributions	1–10	20%
Quality of Work	1–10	20%
Communication	1–10	20%
Professionalism & Reliability	1–10	20%

Individual project grades may be adjusted based on peer evaluations, GitHub contributions, instructor observations, presentation participation, etc. Students who contribute significantly less than their teammates may receive a reduced individual project grade.

AI Usage Policy

AI tools such as ChatGPT, GitHub Copilot, Claude, Gemini, and similar tools may be used for brainstorming, debugging, research, or generating starter code.

- Clearly identify where AI assistance was used.
- Describe how your team modified or extended AI-generated work.
- Provide references when appropriate.
- Be prepared to explain and defend all submitted work.

Phase 1 – Draft Database Product

Draft doesn't have to be perfect; just complete all sections below and try your best to have a working prototype. Develop a draft website using google sites, weebly.com, wix.com, or any website builder for the Phase 1 Draft Database Product. Provide links on the webpage to the following deliverables below, except for the Peer Evaluation which will be submitted in the comments section of your individual submission.

Deliverable	Description	Points
Phase 1 Draft Presentation	Equally present a 10-minute (8-minute presentation and 2-minute Q&A) during the midpoint of the semester. Sign up for presentations in the appropriate discussion thread in Canvas. Due date in Canvas.	25
Draft About Webpage	Introduce your team, your hypothetical software company, its mission, team roles, the real-world problem your application solves, and the value it provides to users. If appropriate, briefly describe how the project could evolve into a future software product or startup	10

company.

Draft Database Product Overview & Instructions	Overview of your product and user instructions.	10
Draft Blog	Each team member writes at least one paragraph describing the project and why it is unique.	10
Draft Source Code & Executable	GitHub repository, executable, and summary of original code beyond AI assistance.	25
Draft Database Product Document	Technical PDF describing design, implementation, course concepts applied, screenshots, and user guide.	10
Phase 1 Peer Evaluations	Complete peer evaluations of each of your team members in the comments section of your individual submission. Do not place on website.	10

Phase 1 Draft Database Product Total: 100 points

Peer Reviews:

Please review at least 2 classmate's Draft Database Project and provide feedback to improve their Draft Database Project by answering the questions below. (This is a discussion post in Canvas; see Canvas for due date). (30 points)

- Did you find your classmate's Draft Database Product functional and useful? Why or why not? What improvements would you suggest? Rate your classmate's Draft Database Product on a scale of 1 to 10. Please provide reasons for your ratings.

Please reply to at least 2 classmates' posts about your Draft Database Product. This is a discussion post in Canvas; see Canvas for due date (20 points). Note that posts need to be substantive with meaning, and not just I agree. Posts should be at least 3 or more quality sentences.

Phase 1 Peer Reviews Total: 50 points

Phase 2 – Final Database Product

Provide links on the Phase 2 Final Project webpage to the following deliverables below, except for the Peer Evaluation which will be submitted in the comments section of your individual submission.

Deliverable	Description	Points
Phase 2 Final Presentation	Equally present a 10-minute (8-minute presentation and 2-minute Q&A) during the midpoint of the semester. Sign up for presentations in the appropriate discussion thread in Canvas. Due date in Canvas.	25
Final About Webpage	Enhanced About page with revisions based on feedback, including updates to your software company's mission, product vision, and team roles as appropriate.	10
Final Database Product Overview & Instructions	Enhanced overview and instructions.	10
Final Blog	Enhanced blog (minimum three paragraphs per team member).	10
Final Source Code & Executable	Updated GitHub repository, executable, and contribution summary.	25
Final Database Product Document	Enhanced technical PDF with revisions and improvements.	10
Phase 2 Peer Evaluations	Complete peer evaluations of each of your team members in the comments section of your individual submission.	10

Phase 2 Final Database Product Total: 100 points

Peer Reviews:

Please review at least 2 classmate's Final Database Product and provide feedback to improve their Final Database Product by answering the questions below. (This is a discussion post in Canvas; see Canvas for due date). (30 points)

- Did you find your classmate's Final Database Product functional and useful? Why or why not? What improvements would you suggest? Rate your classmate's Final Database Product on a scale of 1 to 10. Please provide reasons for your ratings.

Please reply to at least 2 classmates' posts about your Final Database Product. This is a discussion post in Canvas; see Canvas for due date (20 points). Note that posts need to be substantive with meaning, and not just I agree. Posts should be at least 3 or more quality sentences.

Phase 2 Peer Reviews Total: 50 points

Requirements

- Phase 1 Draft Database Product Project Presentation
- Phase 1 Draft About
- Phase 1 Draft Database Product Overview & Instructions
- Phase 1 Draft Blog
- Phase 1 Draft Database Source Code & Executable
- Phase 1 Draft Database Product Document
- Phase 1 Peer Evaluations
- Phase 1 Peer Reviews

- Phase 2 Final Database Product Project Presentation
- Phase 2 Final About
- Phase 2 Final Database Product Overview & Instructions
- Phase 2 Final Blog
- Phase 2 Final Database Source Code & Executable
- Phase 2 Final Database Product Document
- Phase 2 Peer Evaluations
- Phase 2 Peer Reviews

Things to consider for your Final Project:

Task definition: What does your system do (what is its input and output)? What real-world problem does this system try to solve? Make sure that the project's scope is not too narrow or broad.

The first task you might come up with is to build a basic database management system to track a simple entity, such as products in an inventory. However, this is probably not enough. You should aim for a task that requires more complex operations, such as handling relationships between entities (e.g., users, orders, and products).

An essential part of defining the task is the evaluation. In other words, how will you measure the success of your system? For this, you need to obtain a reasonably sized dataset of example input-output pairs, either from existing sources or by collecting one from scratch. A natural evaluation metric could be the system's efficiency in terms of query performance or its ability to handle multiple concurrent users. The size of the dataset depends on the task at hand.

Infrastructure: To build a sound system, you need to set up the infrastructure. This involves designing the database schema, creating tables, and implementing relationships between entities (e.g., using foreign keys). While infrastructure is necessary, try not to spend too much time on it. You can often use existing database systems or simple frameworks to save time. The key is to focus on solving a real-world problem rather than just implementing database operations.

Approach: Identify the challenges of building the system and the problems in data modeling that you're trying to capture. How should you structure the database (e.g., normalization, indexing, etc.)? There will be many ways to do this, but you should pick the most appropriate strategy and explain how the design addresses the challenges. Consider database optimization techniques like indexing, query optimization, and handling transactions. In addition to your primary approach, you should compare with simple baselines, such as an unoptimized query or a table without normalization. These baselines are important as they provide a reference point to evaluate the efficiency and effectiveness of your design.

Literature review: Have there been other attempts to build similar database systems? Compare your approach with existing work, citing relevant papers or case studies. Discuss how your database design

or model is innovative, how it handles performance issues, and whether it can scale with increasing data size or complexity.

Error analysis: Design a few experiments to show the strengths and weaknesses of your system. For example, test how the system performs under different levels of database traffic or how it scales with larger datasets. Each experiment should have a clear question: How does the database handle large joins? Or, what is the impact of indexing on query performance? Analyze the results and present any surprising findings with data visualizations (e.g., graphs or tables).

Example Strategies for a Database Final Project

A simplified example is here. It is oversimplified and not suitable for a final project, but it is a good example to start with. Here are suggestions for approaching the final project with an example.

Example Strategy A:

- Pick a topic that interests you and is related to databases (e.g., online shopping, library management, hospital patient records, inventory systems, or student registration). We're interested in creating a **Library Management System** for this example.
- Brainstorm tasks that can be tackled using a database system. Ask yourself, "What tasks can be efficiently managed with a database?" or "How can I organize and retrieve data meaningfully?" A good task should balance complexity and feasibility. Let's focus on building a system to track book inventory, manage book checkouts, and handle user borrowing history.
- Define the task clearly. In this case, the task is to create a system that allows librarians to track the availability of books, check out books to patrons, and maintain a record of borrowed books and due dates. Establish the project's key objectives, such as:
 - Tracking book details (title, author, genre, ISBN)
 - Handling user information (borrowers, library members)
 - Managing transactions (checkouts, returns, overdue books)

Additionally, define your evaluation metric. For example, you might measure success by how easily the system handles queries or by how accurately it tracks books and due dates.

- Gather and clean the necessary data. This might involve creating mock data or pulling data from an existing source, like an open dataset or a website (e.g., using an API). In this step, ensure your data is well-organized and normalized. You might need to filter out invalid or incomplete data.
- Implement a basic database schema. Design the database with tables for books, users, and transactions. Establish the relationships between these entities (e.g., foreign keys, primary keys). You might also create views or stored procedures to handle common operations like checking out a book or returning it.
- Implement basic CRUD (Create, Read, Update, Delete) operations. These operations will form the foundation of your system. Start by developing simple SQL queries to insert, update, and retrieve data. For example, you can create a query to search for a book by its title or retrieve a user's borrowing history.
- Optimize and extend the system. Once the basic functionality is implemented, you can focus on improving the system. This may include:

- Adding constraints (e.g., unique user IDs, book availability)
- Implementing indexes for faster searches
- Handling more complex queries (e.g., joining multiple tables to show all overdue books for a user)
- Adding additional features such as reservation functionality, late fees, or email notifications
- Finally, analyze the system's performance. Test how well your database handles different operations, such as multiple concurrent checkouts or large numbers of books. Measure query efficiency, especially for search operations, and approach how the system scales with increased data (e.g., more books and users).

Example Strategy B:

This is a suggestion for approaching the final project with an example.

- Choose a domain that interests you (e.g., e-commerce, healthcare, education, social media). For this example, let's consider a library management system.
- Identify the core entities and relationships in your chosen domain. We might have entities like Books, Members, Loans, and Authors for a library system.
- Define the problem you want to solve. In our library example, we could focus on creating an efficient system for tracking book loans, managing inventory, and providing insights into reading trends.
- Outline the main features of your database system. For instance:
 - Book checkout and return process
 - Member registration and management
 - Book inventory tracking
 - Reporting on popular books and frequent borrowers
- Design your database schema:
 - Create an Entity-Relationship (ER) diagram
 - Define tables, attributes, and relationships
 - Normalize your schema to reduce redundancy
- Implement your database:
 - Choose a Database Management System (DBMS) like MySQL or PostgreSQL
 - Write SQL scripts to create tables and define constraints
 - Populate your database with sample data
- Develop queries and stored procedures to support your features:
 - Write SQL queries for common operations (e.g., checking out a book)
 - Create stored procedures for complex operations (e.g., calculating fines)
 - Implement triggers for maintaining data integrity (e.g., updating book availability on checkout)

- Design and implement a simple user interface:
 - Create forms for data entry and retrieval
 - Develop reports for data analysis
 - You can use tools like PHP, Python with Flask, or Java with JDBC
- Optimize your database:
 - Create appropriate indexes to improve query performance
 - Analyze and optimize slow queries
 - Implement database security measures (e.g., user authentication, access control)
- Test your system:
 - Develop test cases for various scenarios
 - Perform stress testing to evaluate system performance under load
 - Validate data integrity and constraint enforcement
- Document your project:
 - Write a comprehensive report detailing your design decisions
 - Include the ER diagram, schema definitions, and key SQL scripts
 - Provide instructions for setting up and using your system
- Analyze and reflect on your project:
 - Discuss the strengths and limitations of your design
 - Suggest potential improvements or extensions
 - Reflect on lessons learned and challenges faced during development

Libraries:

You are free to use existing tools for parts of your project if you're clear about what you used. When you use existing tools, the expectation is that you will do more in other areas.

Strongly Recommended:

- **SQLite:** A lightweight, self-contained SQL database engine that requires minimal setup and is ideal for small projects.
- **MySQL:** A widely-used open-source relational database known for its ease of use and integration with web applications.
- **Pandas (with SQLite or CSV):** A Python data analysis library that can work with SQLite or CSV files to manage and manipulate tabular data.

Recommended:

- **PostgreSQL:** A powerful, open-source relational database system known for its support for complex queries and data types.
- **MongoDB:** A NoSQL database that stores data in flexible, JSON-like documents, suitable for handling unstructured or semi-structured data.
- **SQLAlchemy:** A Python SQL toolkit and Object-Relational Mapping (ORM) library that simplifies database interactions by abstracting SQL queries.
- **Microsoft Access:** A desktop relational database tool with Microsoft Office, useful for small-scale projects with a visual interface.

Some project ideas:

- **Student Course Registration System:** Design a database to manage student enrollments, course schedules, and grades, including conflict resolution for overlapping classes.
- **Library Management System:** Create a database to track books, borrowers, due dates, and late fees, with features like searching, lending, and returning books.
- **Online Store Inventory and Orders:** Develop a database to manage products, customer orders, inventory levels, and shipping, with integration for order tracking.
- **Hospital Patient Record System:** Build a database to store patient information, medical history, doctor appointments, and treatment records, ensuring data privacy.
- **Movie Rental System:** Design a database for managing movie rentals, customers, inventory, and late return penalties, with a recommendation feature for popular movies.
- **Event Ticket Booking System:** Implement a database for event ticket sales, seat reservations, and customer bookings for concerts or sports events.
- **Employee Payroll System:** Create a database to manage employee records, work hours, salaries, and tax calculations with different pay structures (e.g., hourly, salary).
- **Restaurant Reservation and Ordering System:** Build a database to manage table reservations, menu orders, and billing for a restaurant, including support for special requests.
- **E-Learning Platform Database:** Design a database to manage courses, students, instructors, grades, and content (videos, quizzes), allowing student progress tracking.
- **Hotel Booking Management System:** Develop a database to manage room bookings, check-in/checkout dates, customer information, and payment details for a hotel.

Example datasets:

- Car company database
(https://github.com/dtaivpp/car_company_database/tree/master)
- Awesome Public Datasets: A great variety of datasets.
(<https://github.com/awesomedata/awesome-public-datasets>)